PAUTA EXAMEN **Profesor: Ing. Jorge Morris A.**
Ing. Luis Herrera H.

I. Selección Múltiple. (25 pts.)

1. Un archivo ejecutable corresponde a una serie de:
   A) Instrucciones que se ejecutan y almacenan completamente en la memoria principal del computador.
   B) Datos que el cargador lleva a la memoria principal para ser ejecutadas.
   C) Secciones de código de máquina que el cargador lleva a la memoria principal para ejecutarlas.
   D) Instrucciones que el cargador lleva a memoria secundaria para ser ejecutadas.

2. La fragmentación externa en un ambiente de paginación:
   A) Es negativa.
   B) Es positiva.
   C) Es cero.
   D) Es mayor que cero.

3. Un estado Inseguro en el Sistema podría ser causado por:
   A) Los recursos escasos.
   B) Los Marcos de página.
   C) Un ambiente de Paginación.
   D) Un ambiente de Segmentación.

4. Una diferencia fundamental entre “paginación por demanda” y “paginación por adelantado o paginación anticipada”, es:
   A) La “paginación por demanda” privilegia el espacio en memoria, mientras que la “paginación por adelantado” privilegia el tiempo de acceso.
   B) La “paginación por demanda” privilegia el tiempo de acceso y la “paginación por adelantado” privilegia el acceso a los datos en memoria.
   C) La “paginación por demanda” privilegia el espacio en disco, mientras que la “paginación por adelantado” privilegia el espacio en memoria principal.

D) La “paginación por demanda” mantiene las páginas en disco, mientras que la “paginación por adelantado” las mantiene en memoria.

5. Cuando se desea ejecutar un proceso, se carga en la memoria por swapping. Si este proceso, en lugar de intercambiarse completo, se intercambia por partes, estamos hablando de:

A) Paginación por adelantado.
B) Partición por adelantado.
C) Paginación por demanda.
D) Partición por demanda.

II Aplicación

Problema 1. (20 pts.)
Se tiene tres procesos concurrentes P_1, P_2 y P_3, que utilizan dos tipos de recursos R_1 y R_2, tal que, existen N_1 y N_2 recursos de cada tipo, siendo conocida para cada proceso P_i la necesidad máxima $M_{i,j}$ por cada tipo de recurso R_j. Suponiendo que $N_1 = N_2 = 3$ y $M_{i,j} = 2$ para todo proceso y tipo recurso, entonces:

a) Determine la Matriz de Necesidad por cada proceso.
b) Especifique si el Sistema está en un Estado Seguro para los tres escenarios siguientes, especificados por su Matriz de Asignación $N_{i,j}$, debe justificar su respuesta.

Caso 1	R1	R2
P_1	2	2
P_2	1	1
P_3	0	0

Caso 2	R1	R2
P_1	1	1
P_2	1	1
P_3	1	1

Caso 3	R1	R2
P_1	2	0
P_2	1	1
P_3	0	2

a) Matriz de Necesidad por cada proceso.
5 pts.

Caso 1	R1	R2
P_1	2	2
P_2	2	2
P_3	2	2

b) Especifique si el Sistema está en un Estado Seguro para los tres escenarios siguientes, especificados por su Matriz de Asignación $N_{i,j}$, debe justificar su respuesta.

Caso 1	R1	R2
P_1	2	2
P_2	1	1
P_3	0	0

Caso 2	R1	R2
P_1	1	1
P_2	1	1
P_3	1	1

Caso 3	R1	R2
P_1	2	0
P_2	1	1
P_3	0	2

Los tres casos parten con todos los recursos asignados y ninguno disponible.
10 pts.

Sólo el caso 1 refleja un Estado Seguro, donde P1 tiene inicialmente su demanda satisfecha, la secuencia segura sería:
{P1, P2, P3} y {P1, P3, P2} **5 pts.**

Problema 2. (10pts.)
Calcule el tiempo y número de fallas de página utilizando la cadena referencial en un esquema de tres bloques de memoria, con el algoritmo LRU. Asuma que tpo. De acceso = 0.5 ns y tpo. Transferencia = 0.6 ns.

Cadena: 1, 3, 2, 0, 1, 2, 0, 2, 4, 5, 0, 6, 5, 4, 1, 3, 5, 2, 4, 6

Respuesta:
LRU **5 pts**

1	3	2	0	1	2	0	2	4	5	0	6	5	4	1	3	5	2	4	6
1	1	1	0	0	0	0	0	0	5	5	5	5	5	5	3	3	3	4	4
	3	3	3	1	1	1	1	4	4	4	6	6	6	1	1	1	2	2	2
		2	2	2	2	2	2	2	2	0	0	0	4	4	4	5	5	5	6

Número Fallas de Páginas : 16 **2 pts**
Tiempo de Paginación : 31.9 ns (No contempla tiempo promedio de Ejecución por página.) **2pts**

Página	Entradas	Salidas	Total
0	2	2	4*1.1=4.4
1	3	3	6*1.1=6.6
2	2	1	3*1.1=3.3
3	2	2	4*1.1=4.4
4	3	2	5*1.1=5.5
5	2	2	4*1.1=4.4
6	2	1	3*1.1=3.3

1 pts.

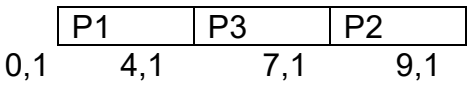
Problema 3.(30 pts.)
Asuma que tiene los siguientes procesos para ejecución:

Proceso	Tpo.Llegada	Tpo. Ráfaga	Prioridad
P1	0,1	4	3
P2	0,8	2	0
P3	0,5	3	2

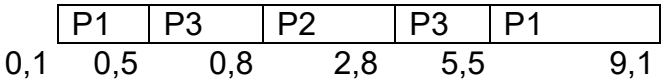
i) Use un diagrama que ilustre la ejecución de estos procesos, usando los algoritmos: FCFS y SJF preemptive.

ii) Calcule el Turnaround time, Waiting time y promedios por cada proceso.

FCFS **(10 pts.)**



SJF Preemptive **(10 pts.)**



0,5 4,0 0,8 3,0
- 0,1 -0,4 -0,5 -0,3
0,4 3,6RR 0,3 2,7RR

P1 P3

Proceso	Tpo.Llegada	Tpo. Ráfaga	FCFS T.T.	FCFS W.T.	SJF P. T.T	SJF P. W.T.
P1	0,1	4	0.0	4,0	9,0	5,4
P2	0,8	2	8,3	6,3	2,0	0,0
P3	0,5	3	6,6	3,6	5,0	2,3
(0,62 pts. C/u)		Promedios :	4,96	4,63	5,33	2,56

Problema 4.(15 pts.)
Un edificio cuenta con un estacionamiento con capacidad limitada para 5 autos. Los residentes llegan a distintas horas y cada uno es representado por un proceso. La entrada y salida deben estar coordinadas, ya que sólo hay una barrera que permite el paso de un vehículo a la vez.

Requisitos:

- Simular múltiples procesos (autos) intentando entrar y salir del estacionamiento. (puede usar lenguaje C o Python)
- La entrada y salida deben estar sincronizadas para evitar colisiones (solo un auto en la barrera).
- Si el estacionamiento está lleno, los autos deben esperar su turno.
- Debe garantizarse que no haya condiciones de carrera, interbloqueos ni inanición.

En Python:

```
import threading
import time
import random

# Capacidad máxima del estacionamiento
MAX_CAPACIDAD = 5
# Semáforo para controlar cuántos autos pueden estar estacionados
capacidad_estacionamiento = threading.Semaphore(MAX_CAPACIDAD)
# Lock para sincronizar la entrada/salida por la barrera
barrera = threading.Lock()

# Cola de prioridad (simplificada)
autos_prioritarios = ["Emergencia", "Discapacidad"]

def auto(nombre, prioridad=False):
    print(f"{nombre} llega al estacionamiento. Prioridad: {prioridad}")

    # Simula espera si no hay espacio, con prioridad básica
    if prioridad:
```

```
        print(f"{nombre} tiene prioridad, intenta acceder primero.")
    else:
        print(f"{nombre} espera turno para ingresar.")

    # Espera por espacio disponible
    capacidad_estacionamiento.acquire()

    # Sección crítica: entrada sincronizada
    with barrera:
        print(f"{nombre} está ingresando...")
        time.sleep(1)  # Simula tiempo en la barrera
        print(f"{nombre} ha ingresado al estacionamiento.")

    # Simula el tiempo de estacionamiento
    tiempo_estacionado = random.randint(2, 5)
    time.sleep(tiempo_estacionado)

    # Salida sincronizada
    with barrera:
        print(f"{nombre} está saliendo...")
        time.sleep(1)
        print(f"{nombre} ha salido del estacionamiento.")

    # Libera espacio
    capacidad_estacionamiento.release()

# Crear autos (procesos simulados)
autos = [
    threading.Thread(target=auto, args=("Auto 1",)),
    threading.Thread(target=auto, args=("Auto 2", True)),  # Prioritario
    threading.Thread(target=auto, args=("Auto 3",)),
    threading.Thread(target=auto, args=("Auto 4",)),
    threading.Thread(target=auto, args=("Auto 5", True)),  # Prioritario
    threading.Thread(target=auto, args=("Auto 6",)),
    threading.Thread(target=auto, args=("Auto 7",)),
]

# Lanzar los hilos
for a in autos:
    a.start()

# Esperar a que todos finalicen
for a in autos:
    a.join()
```

Nota:

- El semáforo controla el número de autos dentro del estacionamiento.
- El Lock (barrera) asegura que solo un auto pase por la entrada/salida a la vez.

En C:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define MAX_CAPACIDAD 5
#define TOTAL_AUTOS 7

sem_t capacidad;
pthread_mutex_t barrera;

void* auto_func(void* arg) {
   int id = *(int*)arg;
   int prioridad = id % 3 == 0; // Simulación básica de prioridad

   printf("Auto %d llega al estacionamiento. Prioridad: %s\n", id, prioridad ? "Sí" :
"No");

   // Intentar adquirir espacio
   sem_wait(&capacidad);

   // Entrada sincronizada
   pthread_mutex_lock(&barrera);
   printf("Auto %d está ingresando...\n", id);
   sleep(1); // Simula tiempo en la barrera
   printf("Auto %d ha ingresado.\n", id);
   pthread_mutex_unlock(&barrera);

   // Estacionamiento
   int tiempo_estacionado = (rand() % 4) + 2;
   sleep(tiempo_estacionado);

   // Salida sincronizada
   pthread_mutex_lock(&barrera);
   printf("Auto %d está saliendo...\n", id);
   sleep(1);
   printf("Auto %d ha salido.\n", id);
   pthread_mutex_unlock(&barrera);
```

```
    // Liberar espacio
    sem_post(&capacidad);

    free(arg);
    pthread_exit(NULL);
}

int main() {
    srand(time(NULL));
    pthread_t autos[TOTAL_AUTOS];

    // Inicialización
    sem_init(&capacidad, 0, MAX_CAPACIDAD);
    pthread_mutex_init(&barrera, NULL);

    // Crear hilos (autos)
    for (int i = 0; i < TOTAL_AUTOS; i++) {
        int* id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&autos[i], NULL, auto_func, id);
    }

    // Esperar que todos los autos finalicen
    for (int i = 0; i < TOTAL_AUTOS; i++) {
        pthread_join(autos[i], NULL);
    }

    // Limpiar recursos
    sem_destroy(&capacidad);
    pthread_mutex_destroy(&barrera);

    return 0;
}
```

Nota:

- **sem_t capacidad limita los autos dentro del estacionamiento.**
- **pthread_mutex_t barrera sincroniza el paso por la entrada/salida**